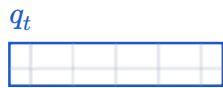


Занятие 1. Цена контекста: от causal attention к линейному состоянию

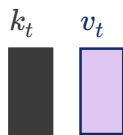
Что именно растёт с длиной контекста — и можно ли заменить историю конечным состоянием?

Автор: Андрей Ильинский, 2026

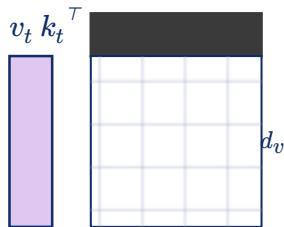
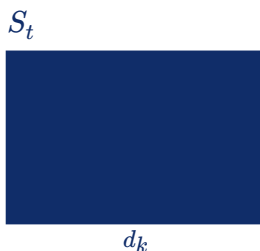
Визуальный словарь курса начинается здесь



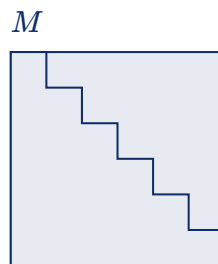
query



key и value

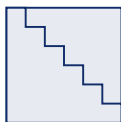
outer product
= одна запись

fixed shape



causal mask

вектор стоимости

9 координат,
порядок не меняется

parallel



recurrent



chunkwise

Стоимость модели — это вектор

$$\mathcal{C} = (M_{\text{weights}}, M_{\text{persistent}}, M_{\text{workspace}}, B_{\text{HBM}}, F, D_{\text{seq}}, N_{\text{FE}}, N_{\text{processed}}, B_{\text{max}}).$$

M_w weights	M_p persistent	M_{ws} workspace	B_{HBM} traffic	F flops	D_{seq} sequential	N_{FE} later	N_{proc} later	B_{max} later
------------------	---------------------	-----------------------	-----------------------------	--------------	--------------------------------	--------------------------	----------------------------	---------------------------

Сегодня активны первые шесть координат; три правые остаются в словаре как задел второй половины курса.

Поместиться в память $\neq$ быть быстрым

$$M_{\text{weights}} + M_{\text{persistent}} + M_{\text{workspace}} + M_{\text{runtime}} \leq M_{\text{GPU}}.$$

говорит
ВМЕСТИМОСТЬ

не говорит
о traffic / latency

GPU memory

weights

persistent state

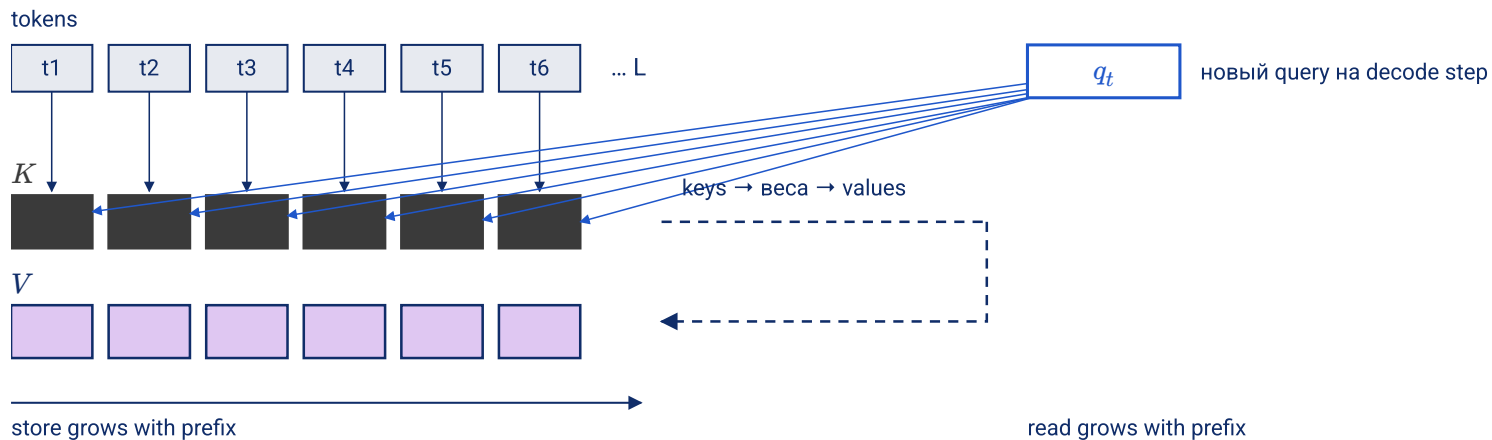
workspace

runtime

KV-cache

Сколько именно мы храним и от чего это зависит?

Почему KV-cache растёт



Сколько scalars добавляет один token

$$2n_{\text{layers}}n_{\text{KV}}d_{\text{head}}.$$

на один token

$$2 \mathbf{L} n_{\text{layers}}n_{\text{KV}}d_{\text{head}}.$$

на весь prefix длины L

Единственное новое — множитель длины: остальные множители фиксированы архитектурой.

KV-cache в bytes

$$M_{KV} = L n_{\text{layers}} n_{KV} d_{\text{head}} s_{\text{dtype}} \times 2.$$

$$M_{KV} = B L n_{\text{layers}} n_{KV} d_{\text{head}} s_{\text{dtype}} \times 2.$$

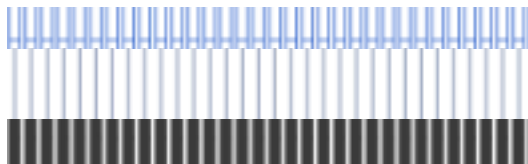
$\times 2$

K + V, не свойство attention

B — параллельные последовательности, а не длина контекста

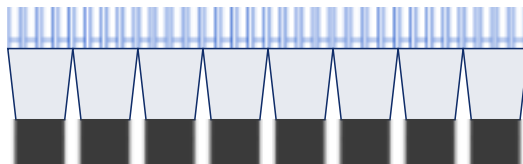
МНА, GQA, MQA меняют число distinct KV-heads

МНА



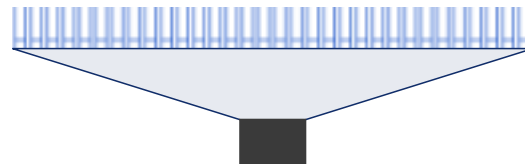
32 query · 32 KV

GQA



32 query · 8 KV, по 4 на группу

MQA



32 query · 1 KV

$$M_{KV} \propto n_{KV}.$$

100k tokens: MHA

$$L = 100000$$

$$n_{\text{layers}} = 32$$

$$n_{\text{KV}} = 32$$

$$d_{\text{head}} = 128$$

$$s_{\text{dtype}} = 2$$

$$B = 1$$

$$M_{\text{KV}} = 100000 \times 32 \times 32 \times 128 \times 2 \times 2 = 52\,428\,800\,000 \text{ bytes.}$$

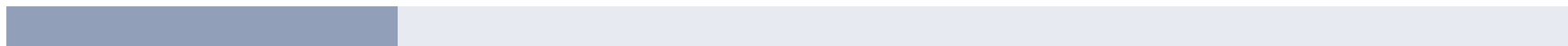
52.43 GB $\approx$ 48.83 GiB

100k tokens: GQA

 $L = 100000$ $n_{\text{layers}} = 32$ $n_{\text{KV}} : 32 \rightarrow 8$ $d_{\text{head}} = 128$ $s_{\text{dtype}} = 2$ $B = 1$

$$M_{\text{KV}} = 100000 \times 32 \times 8 \times 128 \times 2 \times 2 = 13\,107\,200\,000 \text{ bytes.}$$

13.11 GB $\approx$ 12.21 GiB

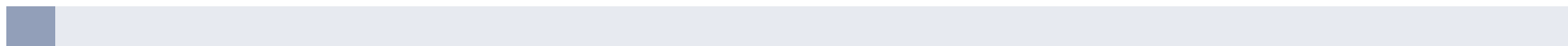


100k tokens: MQA

 $L = 100000$ $n_{\text{layers}} = 32$ $n_{\text{KV}} : 8 \rightarrow 1$ $d_{\text{head}} = 128$ $s_{\text{dtype}} = 2$ $B = 1$

$$M_{\text{KV}} = 100000 \times 32 \times 1 \times 128 \times 2 \times 2 = 1\,638\,400\,000 \text{ bytes.}$$

1.64 GB $\approx$ 1.53 GiB



Один контекст — 32-кратная разница cache

ATTENTION	n_{KV}	RAW KV-CACHE, GB	RAW KV-CACHE, GIB
MHA	32	52.43	48.83
GQA	8	13.11	12.21
MQA	1	1.64	1.53

Длина context одна и та же — различается только число distinct KV-heads.

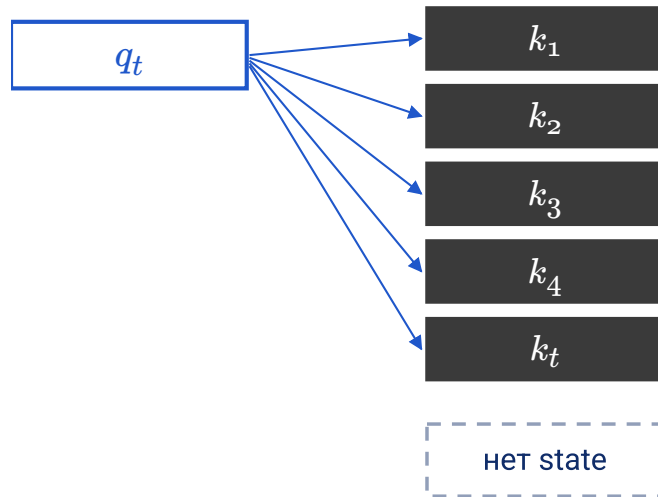
Чтобы агрегировать историю, нужно изменить kernel

Можно ли перестать хранить каждый прошлый key отдельно?

Softmax attention: interaction не факторизована в конечный state

$$\kappa_{\text{sm}}(\hat{q}, \hat{k}) = \exp \left(\frac{\hat{q}^\top \hat{k}}{\sqrt{d_h}} \right).$$

$$y_t = \frac{\sum_{i \leq t} \kappa_{\text{sm}}(\hat{q}_t, \hat{k}_i) v_i}{\sum_{i \leq t} \kappa_{\text{sm}}(\hat{q}_t, \hat{k}_i)}.$$

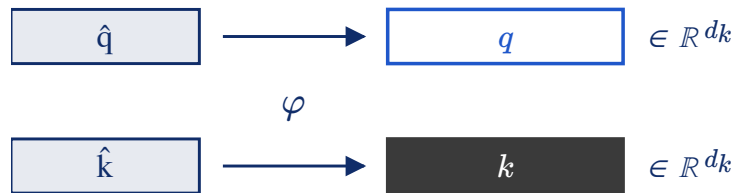


Агрегированного состояния пока нет: каждый прошлый key нужен отдельно.

Меняем класс kernels

$$\phi : \mathbb{R}^{d_h} \rightarrow \mathbb{R}^{d_k}$$

$$\kappa(\hat{q}, \hat{k}) = \phi(\hat{q})^\top \phi(\hat{k}).$$



Это смена mathematical operator, не перестановка softmax.

Базовый causal linear operator

$$y_t = \sum_{i=1}^t (q_t^\top k_i) v_i .$$

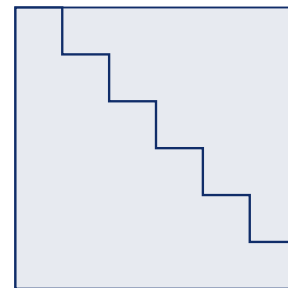
SHAPES

$$q_t, k_t \in \mathbb{R}^{d_k}, \quad v_t \in \mathbb{R}^{d_v} .$$

Parallel form: interactions видны явно

$$M_{ti} = \begin{cases} 1, & i \leq t, \\ 0, & i > t. \end{cases}$$

$$Y = ((QK^{\top}) \odot M)V.$$

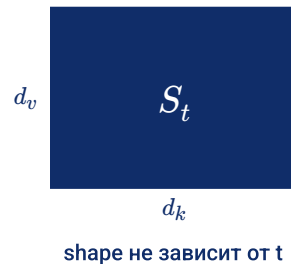


parallel

Сумму можно сгруппировать в matrix state

$$y_t = \left(\sum_{i=1}^t v_i k_i^\top \right) q_t.$$

$$S_t = \sum_{i=1}^t v_i k_i^\top.$$



Из суммы получается recurrence

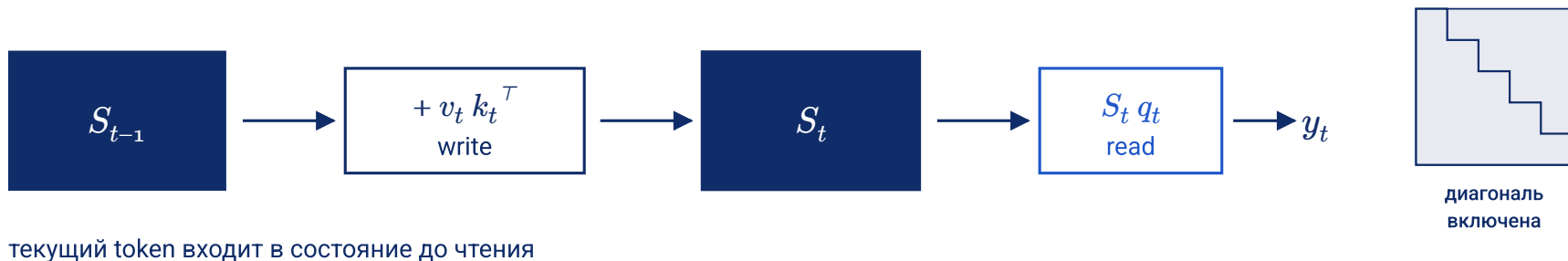
$$S_t = \sum_{i=1}^{t-1} v_i k_i^\top + v_t k_t^\top.$$

$$S_t = S_{t-1} + v_t k_t^\top.$$

$$S_t = S_{t-1} + v_t k_t^\top, \quad y_t = S_t q_t.$$

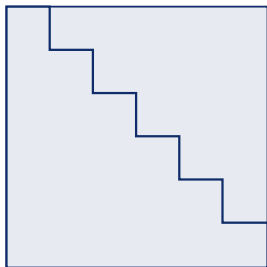
Хранить историю больше не нужно: достаточно последнего состояния.

Inclusive causality: сначала write, потом read



Один operator — три формы

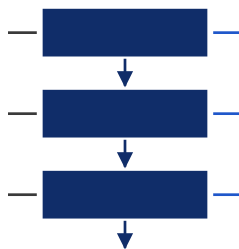
parallel



$T \times T$ causal interaction matrix

=

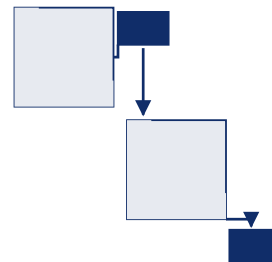
recurrent



цепочка fixed-shape states

=

chunkwise



local blocks + carried state

Chunkwise

Можно ли получить и параллельность обучения, и дешёвый decode?

Chunkwise: разделяем далёкое прошлое и текущий block

$$S_b^{\text{in}} = S_0 + \sum_{i < a_b} v_i k_i^{\top}.$$

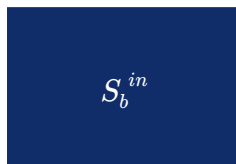
$$y_t = S_b^{\text{in}} q_t + \sum_{i=a_b}^t (q_t^{\top} k_i) v_i.$$

первое слагаемое
past chunks — свёрнуты в state

второе слагаемое
current chunk — явные interactions

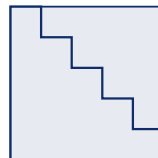
Chunk output

$$Y_b = Q_b(S_b^{\text{in}})^{\top} + ((Q_b K_b^{\top}) \odot M_{c_b}) V_b.$$



incoming fixed state

+

local $c_b \times c_b$

Global $T \times T$ object заменён local interaction внутри chunk.

После chunk переносим только state

$$V_b^\top K_b = \sum_{i \in I_b} v_i k_i^\top.$$

$$S_b^{\text{out}} = S_b^{\text{in}} + V_b^\top K_b.$$

Один write за весь блок: через границу chunk проходит объект фиксированной формы.

Сложность трёх форм

Чем сложность обучения отличается от сложности декодирования?

Один operator — разные costs

ФОРМА	PERSISTENT STATE	PAIRWISE OBJECT	SEQUENTIAL STRUCTURE
Literal parallel	не нужен для полного input	$T \times T$	хорошо parallelizable
Recurrent	$O(d_k d_v)$	нет	$O(T)$ в simple loop
Chunkwise	$O(d_k d_v)$	local $C \times C$	около $O(T/C)$ между chunks

literal parallel

$$O(T^2(d_k + d_v))$$

recurrent

$$O(T d_k d_v)$$

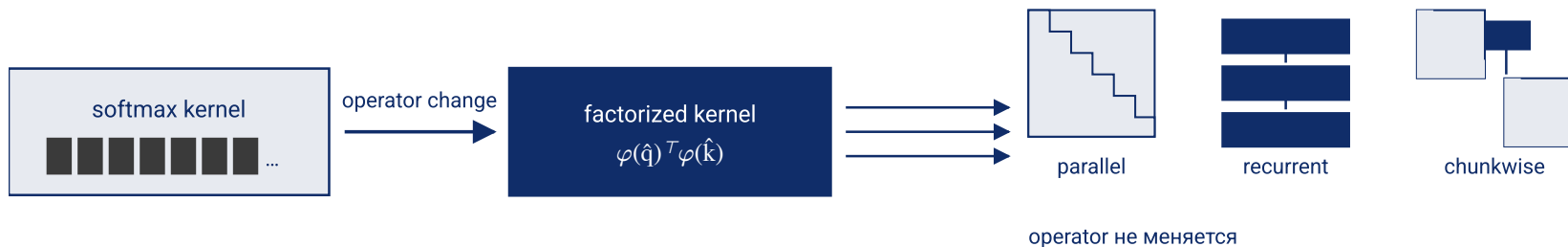
chunkwise

$$O(T d_k d_v + TC(d_k + d_v))$$

Training и decode — разные режимы

РЕЖИМ	PERSISTENT CONTEXT STATE	ARITHMETIC SCALING	ЧТО ОГРАНИЧИВАЕТ
Dense softmax, full sequence	не главный context bottleneck	$O(T^2 d)$	pairwise interactions
Softmax decode + KV-cache	$O(Td)$	$O(Td)$ на новый token	growing cache + reads
Linear recurrent decode	$O(d_k d_v)$	$O(d_k d_v)$ на token	fixed state, no finite capacity
Linear recurrent training	mathematical state fixed	$O(T d_k d_v)$ forward	backward activations + sequential dependence
Linear chunkwise	$O(d_k d_v)$	$O(T d_k d_v + TC(d_k + d_v))$	local matrix + cross-chunk stages

Что именно изменилось



softmax → factorized kernel: меняется operator

parallel ↔ recurrent ↔ chunkwise: меняется evaluation algorithm

Один и тот же оператор — разные алгоритмы вычисления. Из этого не следует, что архитектуры одинаковы.

Fixed shape state — это начало проблемы памяти, а не конец

token-indexed KV-cache



растёт с prefix



shape не меняется

← outer product
← outer product
← outer product ...

$$S_t = S_{t-1} + v_t k_t^\top, \quad y_t = S_t q_t.$$

Следующий вопрос: что происходит, когда разные записи начинают мешать друг другу?